

Lab Report: Week 1 Session A

Environment Setup & First AI-Generated Script

Your Name (Your Student ID)

May 15, 2026

Abstract

This report documents the first hands-on session: configuring an AI-assisted development workflow in OpenCode Desktop, generating a Python script with a structured natural-language prompt, executing it locally, and verifying outputs against the assignment requirements. Evidence includes terminal output (Figure ??), an OpenCode agent conversation capture (Figure ??), and full copies of `hello_ai.py` and `prompt_log.md` in Appendix ??-??.

1 Introduction

The laboratory introduced a basic collaboration pattern with a coding agent: formulate a clear task, review generated code, run it in a terminal, and record prompts and verification steps. The exercise focused on rainfall data: synthesize a CSV with columns `date`, `rainfall_mm`, and `location`; read the file with `pandas`; compute total and average rainfall; and print a short summary report.

2 Objectives

The session objectives (from the course guide) were to:

- Set up the development environment (OpenCode Desktop with OpenCode Zen provider).
- Generate an initial Python script using AI assistance.
- Navigate the IDE (explorer, editor, agent chat, terminal).
- Practice documenting prompts and outcomes in a prompt log.

3 Methods

3.1 Software and layout

Work was organized under `.../Downloads/SoftwareDevelopment/ai_water_lab/week1_session_a/` (ellipsis stands for your home or project root). OpenCode Desktop (v1.14.50) was used with a connected **OpenCode Zen** provider and a general-purpose coding model. Python 3 was invoked as `python3`; the script depends on `pandas` and `numpy`.

3.2 Prompt used (verbatim)

The following prompt was sent to the agent:

I am a water resources student. Please write a Python script that:

1. Creates a sample CSV file with columns: `date`, `rainfall_mm`, `location`

2. Reads the CSV file using pandas
3. Calculates total and average rainfall
4. Prints a summary report

Include comments explaining each step.

3.3 Agent output and local adjustments

The agent produced a working script, initially saved as `rainfall_analysis.py`. The implementation:

- Writes `rainfall_data.csv` with 90 records (30 days $\times$ three stations: `Station_A`, `Station_B`, `Station_C`), using an exponential distribution for synthetic daily depths (seeded for repeatability).
- Reads the CSV with `pd.read_csv`, aggregates by location with `groupby().agg()` for sum and mean, and prints a formatted report including date span, per-station totals and average daily rainfall, overall mean, and maximum daily value.

To match the submission filename in the lab guide, the same code was saved as `hello_ai.py`. A separate `prompt_log.md` records the prompt, summary, verification, and changes (full text in Appendix ??).

Figure ?? shows a screenshot of the OpenCode session (prompt and agent response context).

4 Results

Running `python3 hello_ai.py` completed without errors. The console reported creation of `rainfall_data.csv` with 90 records, previewed the first rows, and printed per-station totals (e.g. `Station_A`: 128.4 mm total; 4.28 mm/day average), overall mean rainfall (4.68 mm/day), and maximum daily rainfall (21.7 mm). Figure ?? shows the captured terminal output.

```
mahmudul-robotics@robotics-ai:~/Downloads/Software Development/ai_water_lab/week1_session_a$ python3 hello_ai.py
Created rainfall_data.csv with 90 records

First 5 rows of the dataset:
   date  rainfall_mm location
0 2024-01-01         2.3 Station_A
1 2024-01-01        15.1 Station_B
2 2024-01-01         6.6 Station_C
3 2024-01-02         4.6 Station_A
4 2024-01-02         0.8 Station_B

=====
RAINFALL SUMMARY REPORT
=====
Date range: 2024-01-01 to 2024-01-30
Total records: 90

Location: Station_A
Total rainfall:    128.4 mm
Average daily:     4.28 mm/day

Location: Station_B
Total rainfall:    155.5 mm
Average daily:     5.18 mm/day

Location: Station_C
Total rainfall:    137.4 mm
Average daily:     4.58 mm/day

Overall average rainfall: 4.68 mm/day
Maximum daily rainfall:   21.7 mm
mahmudul-robotics@robotics-ai:~/Downloads/Software Development/ai_water_lab/week1_session_a$
```

Figure 1: Terminal output after executing `hello_ai.py`: CSV creation, data preview, and rainfall summary report.

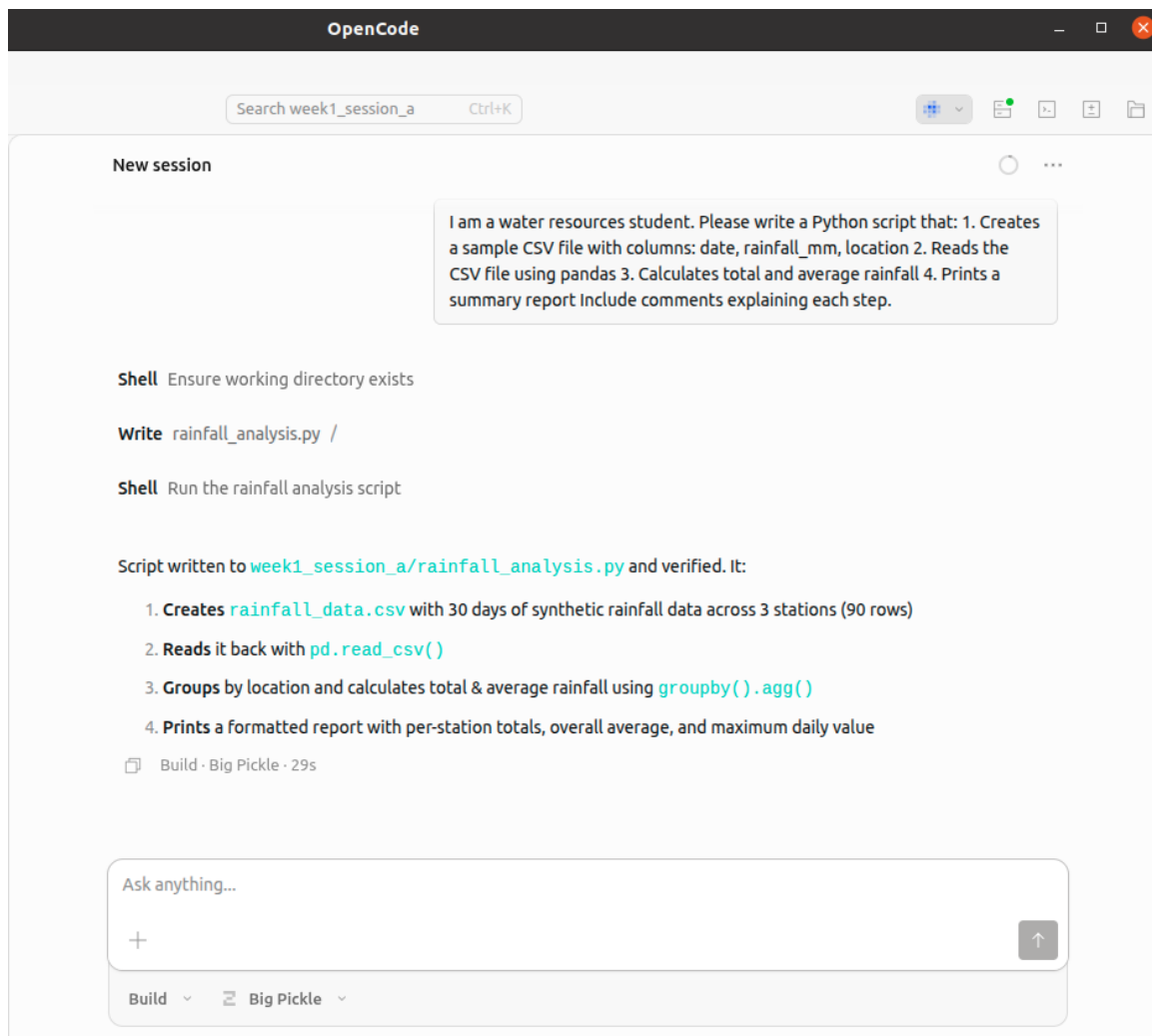


Figure 2: OpenCode Desktop: agent conversation showing the lab prompt and generated solution context.

5 Analysis and validation

Structural checks: The CSV contains the required columns. The script separates creation, read, aggregation, and reporting, with inline comments as requested.

Numerical plausibility: Per-station totals are consistent with summing 30 daily values per station; reported averages equal total divided by 30 for each station. The overall mean is the mean across all 90 rows. Values are non-negative and within a plausible range for synthetic daily depths.

Limitations: Data are synthetic, not observed gauge records, so hydrological conclusions are illustrative only. Random draws could occasionally produce unusually wet or dry spells; the fixed seed improves reproducibility for grading and debugging.

Process note: The agent chose a different default filename than the lab handout: it wrote `rainfall_analysis.py` first. The submission uses `hello_ai.py`. For future labs, ask explicitly for the output filename in the prompt to avoid renaming.

6 Problems encountered

No runtime failures were observed after dependencies were available. The only corrective step was aligning the script filename with the submission checklist by copying to `hello_ai.py`.

7 Lessons learned

Clear, numbered requirements in the prompt produced runnable code with minimal iteration. Human review still mattered for *deliverable alignment* (file naming) and for confirming that reported statistics match the generated table. Keeping a contemporaneous prompt log makes it straightforward to write this report and to evidence the AI workflow for assessment.

8 Conclusion

The session objectives were met: the environment was used end-to-end, a rainfall reader script was generated with AI assistance, executed successfully, and validated at a basic level. This PDF bundles narrative with evidence (Figures ??–??) and full deliverable sources in the appendices. The course portal may still require separate uploads of `hello_ai.py`, `prompt_log.md`, and image files per the lab guide.

A Submitted script: `hello_ai.py`

`hello_ai.py` (full file)

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta

# Step 1: Create a sample CSV file with rainfall data
# Columns: date (YYYY-MM-DD), rainfall_mm (daily precipitation), location (station name)
np.random.seed(42)
start_date = datetime(2024, 1, 1)
locations = [Station_A, Station_B, Station_C]
records = []
```

```

for i in range(30):
    date = (start_date + timedelta(days=i)).strftime('%Y-%m-%d')
    for loc in locations:
        rainfall = round(np.random.exponential(scale=5.0), 1)
        records.append([date, rainfall, loc])

df_sample = pd.DataFrame(records, columns=[date, rainfall_mm, location])
csv_path = rainfall_data.csv
df_sample.to_csv(csv_path, index=False)
print(fCreated_{csv_path}_with_{len(df_sample)}_records\n)

# Step 2: Read the CSV file into a pandas DataFrame
df = pd.read_csv(csv_path)
print(First_5_rows_of_the_dataset:)
print(df.head(), \n)

# Step 3: Calculate total and average rainfall per location
summary = df.groupby(location)[rainfall_mm].agg([sum, mean]).round(2)
summary.columns = [total_rainfall_mm, avg_daily_rainfall_mm]

# Step 4: Print a formatted summary report
print(= * 50)
print(RAINFALL_SUMMARY_REPORT)
print(= * 50)
print(fDate_range:{df['date'].min()}to{df['date'].max()})
print(fTotal_records:{len(df)}\n)

for loc in summary.index:
    print(fLocation:{loc})
    print(fTotal_rainfall:{summary.loc[loc,'total_rainfall_mm']}_mm)
    print(fAverage_daily:{summary.loc[loc,'avg_daily_rainfall_mm']}_mm/day\n)

print(fOverall_average_rainfall:{df['rainfall_mm'].mean():.2f}_mm/day)
print(fMaximum_daily_rainfall:{df['rainfall_mm'].max():.1f}_mm)

```

B Submitted prompt log: prompt_log.md

prompt_log.md (full file)

```

# Prompt Log - Week 1 Session A

## Exercise 2: Rainfall Data Reader

**Date:** 2026-05-15

**Prompt Used:**

I am a water resources student. Please write a Python script that:
1. Creates a sample CSV file with columns: date, rainfall_mm, location
2. Reads the CSV file using pandas
3. Calculates total and average rainfall
4. Prints a summary report
Include comments explaining each step.

**AI Output Summary:**

```

OpenCode generated a script (initially saved as `rainfall\_analysis.py`) that creates
↳ `rainfall\_data.csv` with 90 rows (30 days 3 stations), reads it with `pandas`,
↳ groups by `location` to compute total and average daily rainfall per station, and
↳ prints a formatted summary including date range, per-station totals/averages, overall
↳ mean rainfall, and maximum daily rainfall.

****Verification:****

- Did the code run successfully? ****Yes**** `python3 hello\_ai.py` completed without errors;
↳ `rainfall\_data.csv` was created.
- Were the calculations correct? ****Yes**** totals are sums of `rainfall\_mm` per location
↳ over 30 days; averages match total 30 per station; overall mean matches the mean of
↳ all 90 values (spot-check or recalculate in a notebook if required for your report).
- Any errors or issues? ****None at runtime.**** The agent used filename
↳ `rainfall\_analysis.py` instead of the handout name `hello\_ai.py`; the same code was
↳ copied to `hello\_ai.py` for submission.

****Changes Made:****

- Renamed / duplicated the agent output to `hello\_ai.py` to match the lab submission
↳ requirement.

****Lessons Learned:****

(Edit this in your own words, e.g.:) Specific prompts produce runnable starter code
↳ quickly, but output should always be reviewed (filenames, paths, column names) and
↳ checked against the assignment deliverables before submission.